

Coinductive Symmetric Homomorphism Reinforcement Learning: Optimality as Structure Preservation

Anonymous Author(s)

Anonymous Institution

Abstract. We define a typable optimality criterion based on coinductive preservation of action dominance: an action is better than another if it leads to a strictly better future in the coinductive sense. We realize this criterion through homomorphic rankings—permutations of the action set that preserve the coinductive ordering on successor value streams.

A natural strengthening additionally requires immediate rewards to respect the ranking. These two conditions decompose cleanly; the homomorphism criterion alone is strictly more general and correctly ranks actions in sacrifice patterns where discounted return fails. We give a constructive procedure—the Finder together with violation-correction learning—that computes rankings provably satisfying the criterion. All results are machine-verified in Agda, including monotonic convergence of learning, instant adaptation, stochastic lifting via the Giry monad, and scalable verification on large combinatorial domains with zero postulates.

Keywords: Reinforcement learning · Coinduction · Symmetric group · Formal verification · Agda · Optimality · Stochastic dominance

1 Motivation: The Limitations of Scalar Returns

Reinforcement learning (RL) studies agents that learn to make sequential decisions by interacting with an environment [11]. At each discrete time step, the agent observes a *state*, selects an *action*, and receives a numerical *reward* that signals how good the outcome was. The agent’s goal is to find a *policy*—a rule mapping states to actions—that collects as much reward as possible over time.

The question is how to define “as much reward as possible.” The dominant answer in modern RL is the *expected discounted return*: given a policy π , the agent averages over all trajectories τ it might experience:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where r_t is the reward at time t and $\gamma \in [0, 1)$ is the *discount factor*. The discount makes future rewards exponentially less valuable: a reward 10 steps ahead counts for only γ^{10} of its face value. This ensures the sum converges and makes the problem mathematically tractable.

The result is a single scalar. A policy is optimal when it maximizes this number. This formulation—rooted in Bellman’s dynamic programming principle [1] and operationalized by algorithms like Q-learning [15], policy gradient methods, and deep RL [7]—has driven remarkable successes, from game playing to robotics.

1.1 The Problem: Temporal Structure Loss

Scalar return collapses the temporal structure of reward sequences. Two futures can have similar discounted sums while being structurally very different:

Future	Reward Sequence	Sum ($\gamma=0.9$)
A: “Win then die”	$[10, -5, 0, 0, \dots]$	$10 - 4.5 = 5.5$
B: “Struggle then thrive”	$[0, 5, 0, 0, \dots]$	$0 + 4.5 = 4.5$

A scalar optimizer prefers A, yet B is structurally superior—it avoids catastrophic failure and reaches a productive state. This is **value aliasing**: structurally different futures map to the same scalar. This formulation introduces several artifacts:

- **Discount factor (γ):** An extrinsic parameter that undervalues delayed rewards and can make the optimal policy discontinuous in γ .
- **Instability:** Q-learning’s bootstrapping of scalar estimates leads to the deadly triad (function approximation + bootstrapping + off-policy learning) [11].
- **Credit assignment:** Rewards at large t are effectively zero under discounting, crippling long-horizon learning.

These are not bugs—they follow directly from compressing sequences into scalars. When optimal behavior requires paying a cost now for a benefit later, discounting systematically distorts the decision.

1.2 Our Approach

We instead define optimality via *structure preservation*. We keep reward sequences as infinite streams and rank actions by whether they lead to successor states whose optimal futures dominate others at every depth under a coinductive ordering. The central question is what structure to preserve and how to do so formally.

2 Structural Optimality: Successor State Quality

Consider an agent choosing between two actions at a state. Action a gives an immediate reward of 3 but leads to a dead-end state (reward 0 forever). Action b gives immediate reward 0 but leads to a productive state (reward 5 forever).

A scalar return optimizer with low discount factor might prefer a . But structurally, b is the better action—it leads to a *better state*. The immediate reward is a one-time cost; the successor state quality compounds forever.

This motivates our central criterion:

An action is ranked higher because it leads to a successor state of higher quality. A state is of higher quality when acting optimally from it yields higher rewards at every future depth—which itself depends on the quality of states reachable from there, unfolding coinductively.

The ranking tracks which actions lead to *success*—both in the sense of successor state and in the evaluative sense.

3 Formal Definition

3.1 The Order Homomorphism

We now make the intuition from Section 2 precise. The formalization requires three ingredients; we introduce each in turn.

Actions and rankings. Let $\mathcal{A}$ be the finite set of actions available to the agent. At each state s , the agent maintains a *ranking*—a total preorder $\leq_a$ on $\mathcal{A}$ —expressing which actions it considers at least as good as which others. If $a \leq_a b$, the agent regards b as at least as good as a at state s .

The object of interest is the *entire* order on $\mathcal{A}$, not merely its maximum. A total preorder commits the agent to comparing *every* pair of actions—including the two *worst* ones—rather than only separating a top choice from the rest. A strict total ranking is precisely an element of the symmetric group $S_{|\mathcal{A}|}$ acting on the actions: the agent selects one ordering among all $|\mathcal{A}|!$ permutations, whereas an argmax commitment leaves that choice almost entirely underdetermined. This full-ranking stance is foundational—it is the sense in which the homomorphism is *symmetric*: optimality is judged against the whole permutation structure on actions, not a single distinguished action. An agent that has genuinely understood an environment can say which of even its two worst options is less bad; an argmax-centric agent cannot. The commitment is not merely conceptual: it is what makes

learning correct *every* violated pair, including among the worst (§16), and what makes adaptation to a lost top action an $O(1)$ lookup of the next-best (§17).

Successor states. When the agent takes action a at state s , the environment transitions to a new state and emits a reward. We write $step(s, a)$ for this transition, which produces a pair: the successor state $next(s, a)$ and the immediate reward $r(s, a)$. The successor state is where the agent *ends up*—the starting point of the rest of its life.

Value streams. From any state s' , the agent can act optimally into the indefinite future, collecting a reward at each time step. Rather than collapsing this sequence into a scalar, we keep it as an infinite stream:

$$value(s') = [v_0, v_1, v_2, \dots]$$

where v_t is the optimal reward achievable at depth t from s' . This stream retains the full temporal structure that a scalar return discards. (The precise construction is given in §4.)

Comparing streams. To say one state is “better” than another, we compare their value streams *pointwise*. We write $x \leq_s y$ when stream x is dominated by stream y at every position: for all $t = 0, 1, 2, \dots$, the t -th element of x is $\leq_r$ the t -th element of y . This is a strong condition: the better stream must be at least as good at every single future depth, not just on average or in total.

With these ingredients in hand, the definition is a single sentence:

Definition 1 (CoinductiveHomomorphism — Successor Condition). *The ranking $\leq_a$ satisfies the **CoinductiveHomomorphism condition** at state s if:*

$$a \leq_a b \implies value(next(s, a)) \leq_s value(next(s, b))$$

In words: if the ranking says b is at least as good as a , then the state reached by b must have a value stream that dominates the value stream of the state reached by a —at every future depth, without exception.

The definition says nothing about the immediate rewards $r(s, a)$ and $r(s, b)$. It judges actions entirely by *where they lead*. An action that pays a cost now but reaches a superior state is correctly ranked, because the successor state’s value stream—reflecting the entire optimal future from that point onward—is what matters.

Why “homomorphism”? The map $a \mapsto value(next(s, a))$ sends each action to a stream. The definition requires this map to be *order-preserving*: if $a \leq_a b$ in the action ranking, then $value(next(s, a)) \leq_s$

$value(next(s, b))$ in the stream ordering. An order-preserving map between ordered sets is called a *homomorphism*—it preserves the structure. The qualifier “coinductive” refers to the target: the stream ordering $\leq_s$ unfolds to arbitrary depth, verified one position at a time, without bound—an infinite tower of commuting diagrams:

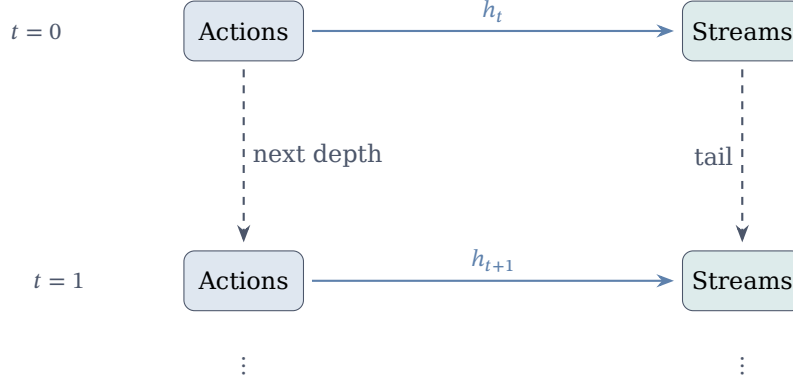


Fig. 1. The coinductive tower: h_t maps each action to its successor-value stream at depth t . Ranking preservation ($a \leq_a b \Rightarrow h_t(a) \leq_s h_t(b)$) must hold at every level, verified by unfolding one tail at a time.

This is a **strategic** evaluation of actions: it assesses the absolute potential of the successor state, allowing for immediate tactical sacrifices.

3.2 The Action-Value Specialization

The CoinductiveHomomorphism condition evaluates successor state quality—the agent’s future *after* the transition. A natural strengthening evaluates the *entire* consequence of an action, including the immediate reward it produces.

Given action a at state s , define the *action-value stream*:

$$action\text{-}value(s, a) = [r(s, a), v_0, v_1, v_2, \dots]$$

Its head is the immediate transition reward; its tail is the value stream of the successor state. The action-value stream is a richer object: it records both what the action pays now and what it sets up for the future.

Definition 2 (CoindHomo — Action-Value Condition). The ranking $\leq_a$ satisfies the **CoindHomo** (action-value) condition at state s if:

$$a \leq_a b \implies action\text{-}value(s, a) \leq_s action\text{-}value(s, b)$$

Because $\leq_s$ compares streams pointwise, this bundles *two* requirements into one: the immediate rewards must compare correctly (the head), *and* the successor state qualities must compare correctly (the tail).

This is a **tactical** evaluation: it demands that the better-ranked action wins both the immediate payoff and the long-term future. When both conditions hold, the ranking is unassailable. But when the optimal action sacrifices immediate reward for a superior successor—the pattern from Section 2—the head condition breaks, and this definition cannot express the correct ranking.

3.3 Decomposition

The relationship between the two definitions is precise:

Theorem 1 (Decomposition). *The action-value condition decomposes into two independent conditions:*

1. **Successor quality** (CoinductiveHomomorphism): $a \leq_a b \implies \text{value}(\text{next}(s, a)) \leq_s \text{value}(\text{next}(s, b))$
2. **Immediate reward compatibility**: $a \leq_a b \implies r(s, a) \leq_r r(s, b)$

Either condition can hold independently. Together, they are equivalent to the action-value condition.

The first condition is CoinductiveHomomorphism—the strategic evaluation. The second is a purely local constraint on immediate rewards. The action-value condition is their conjunction: it insists on tactical *and* strategic soundness simultaneously.

When immediate rewards are aligned with the ranking—as they are in environments with uniform step costs, shaped rewards, or goal-terminal structure—the second condition holds for free, and the two definitions coincide. The gap opens only in environments where the optimal action has a worse immediate reward than a suboptimal alternative: the sacrifice pattern.

As sets of admissible rankings the relationship is a strict containment, $\text{CoindHomo} \subsetneq \text{CoinductiveHomomorphism}$: uniform-step-cost environments lie in the inner set, while sacrifice-now-gain-later environments lie in the gap.

4 The Agda Core

We now present the verified implementation. Every code listing shown below is type-checked by Agda; the snippets are rendered directly from the checked source.

The Core module is parameterized by the environment:

4.1 Value and Action-Value Streams

The key construction: value is an infinite stream obtained by tabulating finite-horizon solutions.

```

solve : State → ℕ → Reward
solve s zero = max-list (map (λ a → proj2 (step s a)) all-actions)
solve s (suc n) = max-list (map (λ a → solve (proj1 (step s a)) n) all-actions)

value : State → StreamR
value s = tabulate (solve s)

action-value : State → Action → StreamR
head (action-value s a) = proj2 (step s a)
tail (action-value s a) = value (proj1 (step s a))

```

`solve s n` computes the maximum reward achievable *exactly* n steps from now, independently maximizing the action at every intermediate step. Crucially, the action sequence that maximizes the reward at depth n may differ from the one that maximizes at depth m . The resulting value stream is therefore *not a trajectory*—no single action sequence produces it. It is the pointwise upper envelope of all possible trajectories: the agent’s **capability profile** from state s , recording the best it *could* achieve at each future depth.

`CoinductiveHomomorphism` compares these capability profiles. When state s_b ’s profile dominates s_a ’s at every depth, the agent prefers any action that reaches s_b —even if that action sacrifices immediate reward. The sacrifice is invisible in the profile: it only records what the agent *can* reach, not the cost of getting there.

The action-value of action a at state s is a stream whose head is the immediate reward and whose tail is the capability profile of the successor state.

4.2 Coinductive Stream Ordering

```

record _≤s_ (x y : StreamR) : Set where
  coinductive
  field
    head≤ : head x ≤r head y
    tail≤ : tail x ≤s tail y

open _≤s_ public

```

A proof of $x \leq_s y$ is an infinite witness: at every time step, the heads compare correctly.

4.3 Why Propositions Instead of Booleans?

The reward ordering $\leq_r$ lives in `Set`—it is a *proposition*, not a Boolean decision procedure. This is a deliberate design choice:

- **Booleans are blind:** `true` carries no information about *why* the comparison holds. To use a Boolean result in a proof, a separate soundness lemma is required.
- **Propositions carry evidence:** A value of type $r_1 \leq_r r_2$ is the proof—no lemma needed.
- **Decidability bridges both:** `Dec (r1 ≤r r2)` is either `yes p` (with proof `p`) or `no ¬p` (with refutation).

This separation threads through the entire architecture: the Core module uses propositions for proofs; the Finder uses Booleans for speed; the Environment Class infrastructure bridges them via `Dec`.

4.4 CoindHomo: The Action-Value Condition

The `CoindHomo` record is the Agda encoding of the Action-Value Coinductive Homomorphism (Definition 2):

```
record CoindHomo : Set1 where
  field
    _≤a_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤a_ s a b →
      action-value s a ≤s action-value s b

open CoindHomo { {...} } public
```

The record has two fields. The first, $\leq_a$, is the ranking itself—a ternary relation indexed by state, taking two actions and returning a type (a proposition, per §4.3). The second, `preserves`, is the proof obligation: for any actions a, b and state s , if the ranking says $a \leq_a b$, then the full action-value stream of a is coinductively dominated by that of b . Because $\leq_s$ checks both head and tail, this bundles two requirements: the immediate reward must compare correctly (head), and the successor state quality must compare correctly (tail).

4.5 CoinductiveHomomorphism: The Successor Condition

The `CoinductiveHomomorphism` record is the Agda encoding of Definition 1. The successor map extracts the state reached by an action:

```
next : State → Action → State
next s a = proj1 (step s a)
```



```

successor-value : State → Action → StreamR
successor-value s a = value (next s a)

```

`successor-value s a` is the optimal value stream from the state reached by taking action a at state s . It is definitionally equal to `tail(action-value s a)`.

```

record CoinductiveHomomorphism : Set1 where
  field
    _≤a_ : State → Action → Action → Set
    preserves : ∀ a b s → _≤a_ s a b →
      successor-value s a ≤s successor-value s b

```

The `CoinductiveHomomorphism` condition requires only that the ranking preserves successor state quality. The immediate transition reward is decoupled by design.

4.6 The Decomposition

```

CoindHomo→CoinductiveHomomorphism : CoindHomo → CoinductiveHomomorphism
CoindHomo→CoinductiveHomomorphism homo = record
  { _≤a_ = CoindHomo._≤a_ homo
  ; preserves = λ a b s prf → tail≤ (CoindHomo.preserves homo a b s prf)
  }

```

Every `CoindHomo` is a `CoinductiveHomomorphism`: project the tail of the action-value stream dominance. The converse does *not* hold.

```

CoinductiveHomomorphism+Head→CoindHomo :
  (ch : CoinductiveHomomorphism) →
  (∀ a b s → CoinductiveHomomorphism._≤a_ ch s a b →
    proj2 (step s a) ≤r proj2 (step s b)) →
  CoindHomo
CoinductiveHomomorphism+Head→CoindHomo ch head-compatible = record
  { _≤a_ = CoinductiveHomomorphism._≤a_ ch
  ; preserves = λ a b s prf →
    combine-≤s (head-compatible a b s prf)
      (CoinductiveHomomorphism.preserves ch a b s prf)
  }

```

When immediate transition rewards also respect the ranking, `CoinductiveHomomorphism` specializes to `CoindHomo`. This characterizes exactly what `CoindHomo` adds: the head constraint, which is orthogonal to successor state quality.

5 When the Conditions Coincide—and When They Diverge

The decomposition theorem reveals that CoindHomo and CoinductiveHomomorphism differ by exactly one condition: whether immediate transition rewards respect the ranking. When does this matter?

5.1 When the Two Conditions Coincide

In many standard environments the head condition holds for free, so CoindHomo and CoinductiveHomomorphism coincide. This includes environments with uniform step costs, shaped rewards that already guide the agent toward optimality, and goal-terminal settings where the only distinction is reaching the goal versus not. In these cases every CoinductiveHomomorphism is automatically a CoindHomo, and all results proved under CoindHomo apply directly.

5.2 When They Diverge: The Sacrifice Pattern

The two conditions diverge precisely when the optimal action has a *worse* immediate reward than a suboptimal alternative. We call this the **sacrifice pattern**: the agent must accept an immediate cost to reach a superior successor state.

This pattern is invisible both to discounted return (which suppresses future payoffs geometrically) and to CoindHomo (which refuses to rank when the head condition fails). The CoinductiveHomomorphism condition is the first in the framework that fully decouples optimality from immediate reward, judging actions solely by the quality of the states they lead to.

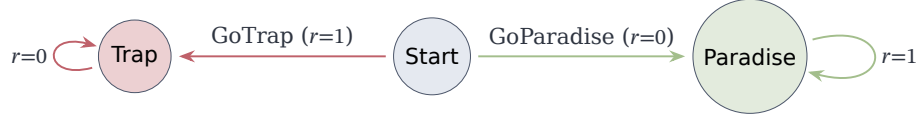
Most standard benchmarks have been shaped so that immediate rewards already align with long-term quality. Environments where immediate rewards actively mislead are regarded as “hard exploration problems” and are typically studied as pathological cases rather than representative ones. As a result, the sacrifice pattern—where the greedy action is wrong—has been quietly engineered out of the environments the field studies most. CoindHomo remains the right tool for the common (aligned) case; CoinductiveHomomorphism extends the framework to the complementary regime without losing anything that CoindHomo provides.

6 Strict Generality: Verified Counterexamples

The implication $\text{CoindHomo} \Rightarrow \text{CoinductiveHomomorphism}$ is strict. We prove this with three verified environments of increasing structural complexity, each exhibiting the sacrifice pattern.

6.1 Sacrifice Now, Gain Later

The minimal separating example has three states and two actions:



GoParadise sacrifices the immediate reward (0 vs 1) but leads to Paradise (reward 1 forever). GoTrap grabs the immediate reward but leads to Trap (reward 0 forever).

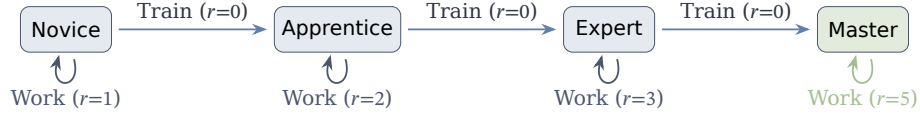
CoinductiveHomomorphism: $value(Trap) = [0, 0, \dots] \leq_s [1, 1, \dots] = value(Paradise)$ $\square$

CoindHomo: Cannot rank $GoTrap \leq GoParadise$ (head requires $1 \leq 0$). Cannot rank $GoParadise \leq GoTrap$ either (tail comparison yields $1 \leq 0$ at the head of the successor streams). The actions are *incomparable*—even though one is clearly better.

Full proof: `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`

6.2 Skill Investment: Multi-Stage Sacrifice

A chain of investment decisions where training sacrifices immediate reward to advance:



The capability profiles (value streams) form a clean progression:

Master: [5, 5, 5, 5, ...]
 Expert: [3, 5, 5, 5, ...]
 Apprentice: [2, 3, 5, 5, ...]
 Novice: [1, 2, 3, 5, ...]

These are *not trajectories*. Consider the Apprentice profile $[2, 3, 5, \dots]$: the 2 at depth 0 comes from Working (best immediate reward), the 3 at depth 1 comes from Training to Expert *then* Working (a different action plan), and the 5 at depth 2 comes from Training twice to Master then Working (yet another plan). Each entry independently records the best the agent *could* achieve at that depth. No single action sequence produces the stream $[2, 3, 5, \dots]$.

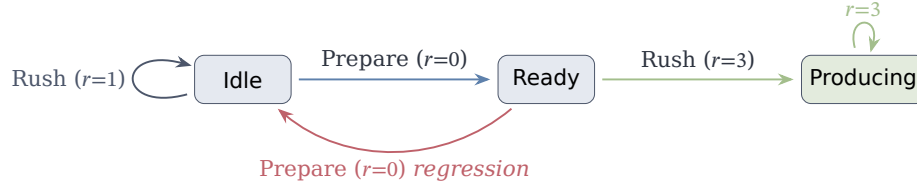
CoinductiveHomomorphism compares these profiles: Train’s successor (Expert, profile $[3, 5, 5, \dots]$) pointwise dominates Work’s successor (Apprentice, profile $[2, 3, 5, \dots]$). The sacrifice—Train’s immediate reward of 0 versus Work’s 2—does not appear in the successor profiles, because the profiles record reachable capability, not incurred cost.

CoindHomo fails at every level: Work’s immediate reward always exceeds Train’s, so the action-value streams are pointwise incomparable.

Full proof: `src/CSHRL/Tasks/Verified/SkillInvestment.agda`

6.3 Preparation Dilemma: Cyclic Dynamics

A structurally different pattern with regression and context-dependent optimal actions:



At Idle, the optimal action is Prepare (sacrifice now for readiness). At Ready, the optimal action flips to Rush (capitalize on preparation). Over-preparing at Ready sends the agent *back* to Idle—a regression penalty.

Value streams: Producing = Ready = $[3, 3, 3, \dots]$, Idle = $[1, 3, 3, \dots]$.

CoinductiveHomomorphism handles the context-dependent ranking: Prepare $>$ Rush at Idle (successor quality improves), Rush $>$ Prepare at Ready (successor quality improves).

CoindHomo cannot rank the actions at Idle in *either* direction:

- Forward (Rush $\leq$ Prepare): head requires $1 \leq 0$ —absurd.
- Reverse (Prepare $\leq$ Rush): head is $0 \leq 1$ (ok), but the tail requires $value(Ready) \leq_s value(Idle)$, which fails at the head: $3 \leq 1$ —absurd.

Full proof: `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`

7 Relationship to Classical Reinforcement Learning

The value streams compared by CoinductiveHomomorphism are computed from the actual environment dynamics by the solve function—they are ground truth, not estimates. If the environment terminates (absorbing states with zero reward), the value streams reflect that. If the environment is infinite, the streams extend accordingly. The horizon is not a parameter of the condition; it is encoded in the dynamics.

7.1 Relationship to Classical Optimality

Both conditions are sound with respect to classical objectives, and each subsumes a different one.

CoindHomo subsumes discounted return. The action-value condition compares the *full* action-value streams, whose head is the immediate reward. Pointwise dominance at every position implies that the cumulative sum—at every finite horizon—is at least as large for the better-ranked action. Since discounted return is a positively weighted sum, action-value dominance implies discounted-return dominance for every $\gamma \in [0, 1)$: any policy optimal under CoindHomo is optimal under every discounted-return criterion.

CoinductiveHomomorphism subsumes successor return. The successor condition compares successor value streams—the accumulated reward *from the state the action leads to*. The verified theorem: The top-ranked action leads to the state with the highest accumulated reward at every finite horizon. This is successor return dominance: the better-ranked action places the agent in a strictly better position.

7.2 CoinductiveHomomorphism Is Sound Where Classical Methods Can Fail

CoindHomo refuses to rank in sacrifice environments, remaining conservative. By contrast, the CoinductiveHomomorphism condition continues to produce correct rankings, while classical discounted methods can actively select the *wrong* action in exactly these cases. To make the contrast concrete, we examine Q-learning—the workhorse algorithm of model-free reinforcement learning [11]. Q-learning maintains a table of *Q-values*, $Q(s, a)$, estimating the expected discounted return of taking action a in state s and acting optimally thereafter. These estimates are updated via the Bellman equation:

$$Q(s, a) \leftarrow r(s, a) + \gamma \cdot \max_{a'} Q(s', a')$$

where $r(s, a)$ is the immediate reward, s' is the successor state, and $\gamma \in [0, 1)$ is the discount factor. The agent then selects the action with the highest Q-value: $\pi(s) = \arg \max_a Q(s, a)$. Under standard conditions, Q-learning converges to the optimal Q-values, and the greedy policy is optimal *for the discounted objective*. The problem, as we now show, is that the discounted objective itself can be wrong.

In sacrifice environments, Q-learning can converge to a suboptimal policy—not because the algorithm fails, but because the discount factor distorts the objective it optimizes. We demonstrate this concretely on the binary sacrifice MDP.

The verified ground truth. The solve function computes the optimal reward at each depth from the actual environment dynamics:

$$\begin{aligned}\text{solve}(\text{Paradise}, n) &= 1 \quad \text{for all } n \\ \text{solve}(\text{Trap}, n) &= 0 \quad \text{for all } n\end{aligned}$$

These are machine-verified facts, not estimates. Paradise yields reward 1 at every future depth; Trap yields 0. The accumulated reward from Paradise at horizon N is N ; from Trap it is 0.

The Q-value computation. The Bellman equation gives Q-learning’s converged values at state Start:

$$\begin{aligned}Q(\text{Start}, \text{GoTrap}) &= \underbrace{1}_{\text{immediate}} + \gamma \cdot \underbrace{0}_{V(\text{Trap})} = 1 \\ Q(\text{Start}, \text{GoParadise}) &= \underbrace{0}_{\text{immediate}} + \gamma \cdot \underbrace{\frac{1}{1-\gamma}}_{V(\text{Paradise})} = \frac{\gamma}{1-\gamma}\end{aligned}$$

For $\gamma < \frac{1}{2}$, we have $\frac{\gamma}{1-\gamma} < 1$, so Q-learning selects GoTrap—the action whose successor state has accumulated reward 0 at every horizon—over GoParadise, whose successor state has accumulated reward N at horizon N .

The discount factor γ geometrically suppresses $V(\text{Paradise})$, which is the only term that carries the evidence of Paradise’s superiority. When γ is small enough, the suppression is sufficient to make a state worth $[1, 1, 1, \dots]$ appear less valuable than a state worth $[0, 0, 0, \dots]$.

The verified conclusion. We prove in Agda (`src/CSHRL/Analysis/QLearningFailure.agda`) the conjunction of two facts:

1. **Tactical:** $r(\text{Start}, \text{GoParadise}) \leq r(\text{Start}, \text{GoTrap})$ — GoTrap has the higher immediate reward.
2. **Strategic:** For every horizon N , the accumulated reward from GoParadise’s successor exceeds GoTrap’s:

$$\sum_{i=0}^{N-1} \text{solve}(\text{Trap}, i) \leq \sum_{i=0}^{N-1} \text{solve}(\text{Paradise}, i)$$

Together: Q-learning with $\gamma < \frac{1}{2}$ selects the action with the higher immediate reward and the strictly inferior successor state. The discount factor is the mechanism by which the tactical advantage overrides the strategic one.

Deeper sacrifice chains. The Skill Investment chain sharpens the point. Q-learning requires $\gamma > (1/5)^{1/3} \approx 0.585$ to identify the optimal training path. Below this threshold, it converges to the policy of working at Novice forever—earning 1 per step while a three-step investment would yield 5 per step permanently. CoinductiveHomomorphism identifies the training path regardless: the successor state quality is strictly better at every depth, and the environment dynamics confirm it.

7.3 CoindHomo as the Conservative Special Case

CoindHomo is best understood as a deliberate conservative restriction of the CoinductiveHomomorphism condition. One might prefer it precisely when its extra demand—that immediate rewards also respect the ranking—is desirable: it bundles tactical and strategic soundness, so its rankings are compatible with existing discounted-return theory and algorithms without further argument. In aligned environments (Section 5), where the two conditions coincide, this comes for free, and CoinductiveHomomorphism inherits CoindHomo’s full subsumption of classical RL.

The restriction has a cost. In sacrifice environments CoindHomo cannot rank the actions at all—the action-value streams are pointwise incomparable—whereas CoinductiveHomomorphism still identifies the better successor state, which the environment dynamics validate. CoindHomo refuses to rank when immediate reward conflicts with the future; CoinductiveHomomorphism resolves the conflict in favor of successor state quality.

8 Stream Isomorphism

We defined stream dominance coinductively: $x \leq_s y$ holds if the heads compare and the tails recursively compare. This definition is equivalent to pointwise dominance at every timestep. The forward direction follows by unfolding the definition. The converse is established by coinduction: if $x_n \leq_r y_n$ for every n , then both $\text{head} \leq$ and $\text{tail} \leq$ hold, hence $x \leq_s y$.

This equivalence has two direct consequences. First, it confirms that the coinductive formulation precisely captures dominance at every future depth. Second, it justifies the completeness of the Finder: because the Finder decides rankings via finite trace comparison, and trace dominance at all depths implies coinductive stream dominance, every ranking returned by the Finder satisfies the coinductive condition.

The isomorphism holds for both successor-value and action-value streams, and therefore supports both CoinductiveHomomorphism and CoindHomo. The full proof is in `src/appendix/StreamIsomorphism.agda`.

9 The Finder Algorithm

The preceding sections established what an optimal ranking is. We now turn to how to *compute* one. The Finder algorithm uses lexicographic trace comparison and requires no discount factor.

9.1 Lexicographic Comparison

Compare traces element-by-element—the first difference decides: Unlike summation (which discards timing information), lexicographic comparison preserves it: a trace $[10, -5]$ dominates $[0, 5]$ because $10 > 0$ at step 0, regardless of subsequent elements.

9.2 Trace Evaluation

Given the comparator, we need the traces themselves. For a deterministic MDP, the trace of action a at state s to depth k is the finite prefix of the reward stream: take action a , record the reward, then act optimally (according to all actions, greedily at each step) for the remaining $k-1$ steps. *best-trace* computes this optimal continuation; *trace-action* prepends the immediate reward of a specific action:

9.3 The Ranking Finder

With traces and a comparator in hand, producing a ranking is straightforward: compute each action’s trace at the desired depth, then sort by lexicographic dominance. The head of the sorted list is the optimal action; the full list is the total ranking used by the adaptation layer (§17).

No discount factor is required. Standard RL needs $\gamma < 1$ to ensure convergence of the sum; here we compare structure directly, and the lexicographic order is well-defined at any depth.

9.4 Serving Both Conditions

The Finder as defined above computes *trace-action*—whose head is the immediate reward and whose tail is the optimal future. This directly serves **CoindHomo**: action-value traces are the finite approximations of action-value streams.

For **CoinductiveHomomorphism**, the relevant comparison is between *successor-value traces*—the optimal future from the state reached by each action, *excluding* the immediate reward. The adaptation is a one-line change: instead of $r :: \text{future}$, use *future* alone. Equivalently, compare $\text{tail}(\text{trace-action } s \ a \ k)$ for each action.

In aligned environments (Section 5), both comparisons yield the same ranking. In sacrifice environments, the action-value trace may be

incomparable (the head reversal prevents lexicographic dominance), while the successor-value trace correctly identifies the better successor state. The Stream Isomorphism (§8) guarantees that trace dominance at all depths implies coinductive stream dominance—for whichever stream variant is used.

10 Environment Classes: Modularity Without Postulates

The Finder produces rankings; the Stream Isomorphism connects them to coinductive stream dominance. But each task still needs its own bridge lemma—a potentially tedious and error-prone process. The **Environment Class** (EC) abstraction eliminates this boilerplate. Rather than proving preservation from scratch for each task, we factor out common structure into reusable modules.

10.1 Parameterized Modules

Without environment classes, every task would need its own Finder, trace-to-stream bridge, and preservation skeleton—duplication that also raises trust questions about which parts are verified. Environment classes instead provide **verified infrastructure** that task instances inherit: a task author fills in a parameter record (ECParameters: types, step function, reward order with decidability and extrema, and finiteness witnesses), and the class automatically supplies the Finder, trace types, stream bridges, and proof scaffolding (full implementation in `src/CSHRL/EnvironmentClass/`).

The fields divide into a *domain* group (State, Action, step), an *algebraic* group (the ordered reward structure $_ \leq_$, $_ \leq? _$, max, bottom), and an *enumeration* group (all-actions, horizon) supplying the finiteness witnesses the Finder needs. From these, the EC derives the trace orderings, the Finder, the ranking relation, the trace-to-stream bridge, and—for specialised ECs—a full CoindHomo.

10.2 What Task Instances Must Provide

For the general **FiniteDeterministicMDP**, a task supplies the domain (State, Action, step) plus a direct proof that the Finder’s ranking preserves action-value ordering. The specialised **CombinatorialPlacementMDP** (for placement problems such as N-Queens, Sudoku, Graph Coloring) reduces this burden further; full implementation in `src/CSHRL/EnvironmentClass/CombinatorialPlacementMDP.agda`.

The three constructors model a placement problem’s life cycle: Ongoing wraps an active configuration, Dead is an absorbing failure (stream $[0, 0, \dots]$), and Solved a completed configuration (stream $[R, R, \dots]$). Any path

reaching Solved thus dominates any path reaching Dead at every timestep—exactly the coinductive order’s requirement. The EC factors preservation into three reusable layers—WithAbsorbingLemmas (absorbing-state domination), WithBinaryStructure (every value is 0 or R , with monotonicity), and WithTraceBridge (horizon sufficiency $\Rightarrow$ full CoindHomo)—so the task author supplies at most four simple ingredients while the machinery handles the finite-trace-to-infinite-stream bridge.

Validation: 8-Queens. The Queens8 module (src/CSHRL/Tasks/Verified/Queens8.agda) instantiates the EC with `--safe` and zero postulates: discharging `solve-horizon-suf` and opening `WithTraceBridge` yields a complete CoindHomo over a combinatorial search space ($8^8 = 16,777,216$ candidate placements) where direct stream enumeration is infeasible. More broadly, verified tasks (TwoState, Queens1, Queens8) carry no postulates; classic tasks may postulate hard-to-formalise properties while keeping the *structural* preservation proof verified; and new tasks inherit all machinery by instantiating the parameter record.

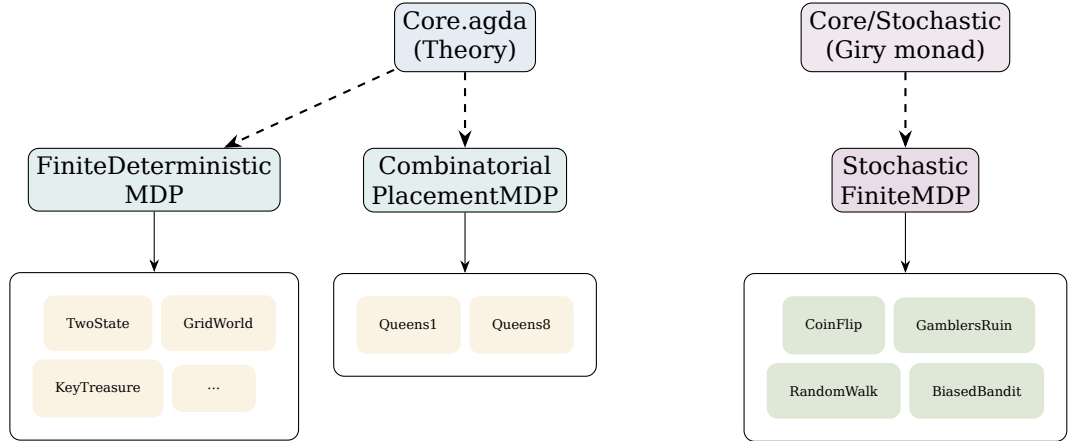


Fig. 2. Environment class hierarchy. Core provides theory; ECs provide reusable verification infrastructure; task groups contain domain-specific instantiations.

11 Stochastic Extension via the Giry Monad

Everything up to this point assumes deterministic environments: each action from a given state leads to exactly one successor state and one reward. But many real-world problems are inherently stochastic—a

robot’s actuators have noise, a card game involves shuffled decks, a medical treatment has variable patient responses. In such environments, the step function returns not a single outcome but a *probability distribution* over outcomes.

The classical approach to stochastic MDPs uses the **Giry monad** [3]: a categorical construction that equips probability measures with a monadic interface (unit, bind), enabling compositional reasoning about probabilistic programs. In full generality, the Giry monad operates over measurable spaces and σ -algebras—far too heavy for constructive, decidable verification in Agda.

Our approach is to use a **finite distribution monad**: a discrete, constructive approximation of the Giry monad that represents distributions as weighted lists. This gives us exact arithmetic (no floating-point rounding), decidable equality, and compatibility with Agda’s termination checker—while being expressive enough to model any finite-state stochastic MDP.

11.1 The Distribution Monad

A distribution over type A is represented as a list of outcome-weight pairs, where weights are natural numbers encoding unnormalized probability mass. For example, a fair coin flip producing Won or Lost is $(\text{Won}, 1) :: (\text{Lost}, 1) :: []$ —each outcome has weight 1 out of total weight 2. This “poor man’s measure theory” suffices for finite MDPs and supports exact arithmetic with no rounding.

The three operations form a monad: `pure` creates a point mass; `scale` multiplies all weights (for conditional branching); and `>=>` implements the law of total probability—each outcome in `d` contributes its weight-scaled continuation. The monad laws are verified in `src/CSHRL/Probability/Finite.agda`.

11.2 Stochastic Step Functions

The key change: the step function returns a distribution over outcomes.

Example (CoinFlip):

11.3 Lexicographic Coinductive Comparison

In stochastic MDPs the “value” at each depth is an *expected* reward. Pointwise comparison $\leq_s$ is too strict here: if action a already wins at depth 0, demanding it also win at every later depth rules out valid rankings. The correct notion is **lexicographic coinductive comparison** ($\leq_{lex}$): compare expected heads, and recurse to tails only when heads are equal. This remains coinductive but lets earlier rewards break ties:

11.4 Verification Pattern

All four verified stochastic examples (CoinFlip, GamblersRuin, RandomWalk, BiasedBandit) share one pattern: one action dominates in expected immediate reward, so $\text{head} \leq$ follows from the ranking hypothesis, while the tail obligation requires the two expected heads to be *equal*—an absurdity (e.g. $0 \equiv 1$ for CoinFlip) discharged by the empty pattern $()$. All four are thus fully verified with no postulates.

11.5 The Stochastic Coinductive Homomorphism

We can now assemble the stochastic analogue of the deterministic CoindHomo record from §4. The structure is identical—a ranking paired with a preservation proof—but the codomain changes: instead of comparing deterministic reward streams with pointwise $\leq_s$, we compare *expected* reward streams with lexicographic $\leq_{\text{lex}}$.

Reading the fields: $_ \leq_a _$ is the ranking, indexed by state, just as in the deterministic case. The `preserves` field says: if the ranking considers b at least as good as a at state s , then the expected action-value stream of a is lexicographically dominated by that of b . The parameter `expected-action-value` is computed by the stochastic Environment Class: it marginalizes the `Dist`-valued step function over all possible successors at each depth, producing an infinite stream of expected rewards.

The full implementation is in `src/CSHRL/Core/Stochastic.agda`, with the stochastic Environment Class in `src/CSHRL/EnvironmentClass/StochasticFiniteMDP.agda`.

11.6 Both Conditions in the Stochastic Setting

Just as the deterministic framework distinguishes CoinductiveHomomorphism (successor state quality) from CoindHomo (full action-value streams), the stochastic extension admits both:

- **StochasticCoindHomo**: preserves expected *action-value* streams lexicographically. This bundles the immediate expected reward and the expected successor quality.
- **StochasticCoinductiveHomomorphism**: preserves expected *successor-value* streams lexicographically. This decouples from immediate reward, allowing sacrifice-pattern reasoning in stochastic environments.

The decomposition theorem (Theorem 1) lifts directly: the stochastic action-value condition decomposes into expected-immediate-reward compatibility plus stochastic successor quality preservation.

Remark 1 (Distributional Hierarchy and CoinductiveHomomorphism). The FOSD and SD[k] extensions (§12–13) are *per-timestep* conditions: they require distributional dominance at each individual depth. This aligns naturally with StochasticCoindHomo, whose action-value stream bundles immediate reward and successor quality at every timestep.

StochasticCoinductiveHomomorphism, by contrast, compares successor-value streams *lexicographically*: dominance is required only at the first depth where expected values differ. Expected successor values may be equal at all finite depths, with the comparison satisfied vacuously. This holistic, stream-level guarantee does not decompose into per-timestep distributional claims, placing lexicographic CoinductiveHomomorphism outside the scope of the per-timestep FOSD/SD hierarchy.

In the deterministic case, $\leq_s$ is unconditionally pointwise (head $\leq$ AND tail $\leq_s$, no equality guard), so the Stream Isomorphism (§8) recovers per-depth dominance. The stochastic split— $\leq_{lex}$ guards tail recursion on head equality—is what creates the gap.

Pointwise dominance implies lexicographic (but not conversely), and the two coincide in the deterministic case. Lexicographic comparison is the correct stochastic notion: when $\mathbb{E}[\text{head}(a)] < \mathbb{E}[\text{head}(b)]$, action b is already established as better and further tail proofs add no information. Classical RL is subsumed by linearity of expectation, $\mathbb{E}[\sum_{t < N} r_t] = \sum_{t < N} \mathbb{E}[r_t]$: comparing partial sums of expected rewards is comparing expected partial sums (proof in appendix/StochasticSubsumption.agda).

The stochastic framework so far compares actions by their expected reward streams. But expected values are themselves a lossy summary—they collapse an entire distribution to a single number. Can we do better? The next two sections develop a hierarchy of distributional comparisons, from FOSD (the strongest) through progressively weaker SD[k] orderings, each capturing a broader class of rational agents.

12 First-Order Stochastic Dominance

The expected-value stochastic pipeline (§11) compares actions by $\mathbb{E}[\text{trace}(s, a)] \leq \mathbb{E}[\text{trace}(s, b)]$. This suffices for expected-value maximizers, but collapses the full reward distribution to a single scalar.

A strictly stronger comparison is **First-Order Stochastic Dominance (FOSD)**: action a FOSD-dominates b at state s iff for all thresholds r and all timesteps n :

$$P(\text{reward}_n(s, a) \leq r) \leq P(\text{reward}_n(s, b) \leq r)$$

i.e., a 's cumulative distribution lies everywhere below b 's. FOSD implies expected-value dominance ($\mu \geq_{\text{FOSD}} \nu \Rightarrow \mathbb{E}[\mu] \geq \mathbb{E}[\nu]$) but not conversely. An action FOSD-dominating another is preferred by *all monotone utility functions*—risk-averse, risk-neutral, and risk-seeking agents agree.

The complete FOSD pipeline is machine-verified across `Probability.FOSD` (distribution level) and `Core.FOSD` (stream level).

At the distribution level, `cdf-weight` computes the CDF of a list-based distribution and the FOSD ordering is pointwise CDF dominance (reflexive, transitive). The bridge `fosd→ev` shows FOSD implies expected-value dominance at equal total weight, and a decidable `fosd?` is proved sound and complete. At the stream level, **Pointwise-FOSD** requires FOSD dominance at every depth n , and its coinductive reformulation `StreamFOSD` is proved equivalent by the **Stochastic Isomorphism**—the distributional analogue of the Stream Isomorphism (§8).

We then define `FOSDCoindHomo`: a ranking preserving `PointwiseFOSD` at every state. Two machine-verified bridges (`pointwise-ev`, `fosd→lex-stream`) establish the key implication `fosd-homo→ev-lex`: every `FOSDCoindHomo` is automatically a `StochasticCoindHomo`, so FOSD verification simultaneously guarantees expected-value optimality *and* dominance for all monotone utilities (full implementation in `src/CSHRL/Probability/FOSD.agda`, `src/CSHRL/Core/FOSD.agda`).

13 The Stochastic Dominance Hierarchy

FOSD provides the strongest distributional comparison: an action FOSD-dominating another is preferred by *all* monotone utility functions. But this strength is also a limitation—many distributions that exhibit a clear “better” relationship fail the FOSD test because their CDFs cross at some threshold.

The **Stochastic Dominance Hierarchy** generalizes FOSD to a parameterized family of orderings, each capturing a broader class of utility functions. The k -th order SD condition integrates the CDF k times; each additional integration “smooths out” CDF crossings:

$$\text{SD}[0] = \text{FOSD} \Rightarrow \text{SD}[1] = \text{SOSD} \Rightarrow \text{SD}[2] = \text{TOSD} \Rightarrow \dots$$

The hierarchy is built on a single operation: **prefix sum** (the discrete analogue of integration). Define `sd-weight 0` as the CDF (`cdf-weight`) and `sd-weight (suc k)` as the prefix sum of `sd-weight k`; the level- k ordering is $\mu \text{SD}[k] \leq \nu = \forall r. \text{sd-weight } k \nu r \leq \text{sd-weight } k \mu r$. Three facts are machine-verified at the distribution level (`Probability.SD`):

- **One-line subsumption** (SD-subsumes): $\mu \text{SD}[k] \leq \nu \Rightarrow \mu \text{SD}[k+1] \leq \nu$, a direct application of prefix-sum monotonicity—giving the full chain $\text{FOSD} \Rightarrow \text{SOSD} \Rightarrow \text{TOSD} \Rightarrow \dots$ in one step.

- **SD-to-EV bridge** ($\text{sd} \rightarrow \text{ev}$): since $\text{sd-weight } k \ d \ 0 \equiv \text{cdf-weight } d \ 0$, $\text{SD}[k]$ implies expected-value dominance at equal total weight, for every k .
- **Strict separation**: SOSD is strictly weaker than FOSD (e.g. $\mu = \{0:2, 2:1\}$, $\nu = \{1:3\}$: FOSD fails at $r=1$, but the integrated CDFs are ordered—confirmed by `refl`).

At the stream level (Core.SD), **PointwiseSD** k requires $\text{SD}[k]$ dominance at every depth, and $\text{SDCoindHomo } k$ is a ranking preserving it. Then $\text{SDCoindHomo } 0$ is FOSDCoindHomo , and sd-homo-subsumes shows a level- k ranking is automatically a level- $(k+1)$ ranking—so a single FOSD verification establishes optimality at *all* levels. The **SD Isomorphism** (sd-isomorphism) lifts the FOSD coinductive/pointwise equivalence uniformly to every k : the proof never inspects the relation on distributions, so the same head/tail decomposition works at all levels, with the Stochastic Isomorphism of §12 as the case $k=0$. Finally, the master bridge sd-homo-ev-lex shows any $\text{SDCoindHomo } k$ yields lexicographic EV-stream dominance: rankings verified at any SD level yield expected-value optimality, while higher levels relax the requirement for broader utility classes.

Remark 2 (Utility Function Classes). The SD hierarchy parameterizes the tradeoff between verification strength and agent generality:

- **SD[0] (FOSD)**: optimal for all monotone utilities (risk-seeking, neutral, and averse).
- **SD[1] (SOSD)**: optimal for all concave monotone utilities (risk-averse agents).
- **SD[2] (TOSD)**: optimal for all utilities with non-increasing absolute risk aversion (prudent agents).
- **SD[k]**: each level captures a broader class, with weaker verification requirements.

When FOSD cannot be established between two distributions (CDF crossing), SOSD may still hold—accepting distributions with the same “integrated tail behavior.” The hierarchy lets the framework verify the strongest relationship that exists, rather than giving up when FOSD fails.

Full implementation: `src/CSHRL/Probability/SD.agda` and `src/CSHRL/Core/SD.agda`.

14 Compositional Ranking Algebra

As environments grow in complexity, monolithic verification becomes impractical. Consider a warehouse robot that must navigate (an MDP

with 20 states) *and* pick items (an MDP with 50 configurations). Verifying the product MDP ($20 \times 50 = 1000$ states, $O(|A|^2)$ pairs per state) is far more expensive than verifying each component independently. The Compositional Ranking Algebra proves that verified rankings *compose*—enabling a “divide and conquer” strategy: verify navigation and picking separately, then combine.

The foundation is that prefix-sum is a *linear operator*, so the $SD[k]$ ordering (built from iterated prefix sums) is closed under distribution concatenation ($SD-++$) and non-negative scaling ($SD-scale$)—both one-line proofs by monotonicity. A `VerifiedRanking` bundles a ranking with a machine-checked proof that it preserves $SD[k]$ at every state and timestep, and the algebra provides five verified operations that combine `VerifiedRankings` with *zero* additional proof burden: hierarchy subsumption ($k \rightarrow k+1$), product composition ($++$ -product), convolution product ($conv$ -product, for additive-reward products, via discrete Abel summation), sum composition (disjoint environments), and scaling. They compose freely—e.g. scaled-product $c \text{ } vr_1 \text{ } vr_2$ —enabling divide-and-conquer verification of multi-component MDPs (full implementation in `src/CSHRL/Probability/Compose.agda`, `src/CSHRL/Core/Compose.agda`).

15 Verified State Abstraction

Everything so far applies to finite MDPs where the state space can be enumerated. But many real-world problems have infinite or very large state spaces—a robot’s continuous position, a game with billions of configurations. Verified state abstraction bridges this gap: verify a ranking on a small, finite *abstract* system, then lift it to the full concrete system.

A `StateAbstraction` bundles three components:

- **project**: $\text{Concrete} \rightarrow \text{Abstract}$ —collapses concrete states to abstract classes.
- **embed**: $\text{Abstract} \rightarrow \text{Concrete}$ —selects a representative for each class.
- **section**: $\text{project} \circ \text{embed} = \text{id}$ —the representatives are consistent.

No finiteness assumption is placed on `Concrete`: it can be $\mathbb{N}$, $\mathbb{R}^n$, or any type in `Set`.

15.1 The Lifting Theorem

The central result is `abstract-lift`:

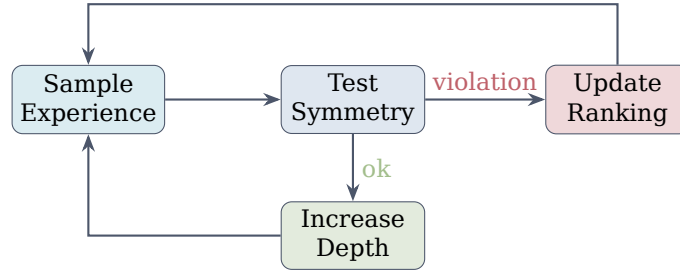
Given a state abstraction and a proof that the marginal reward function is constant within each abstract class (marginal-invariance), any `VerifiedRanking` on the abstract system lifts to a `VerifiedRanking` on the full concrete system.

This is the standard model-irrelevance condition from the state abstraction literature [6], but formalized constructively in Agda with a machine-checked proof. The verification cost is $O(|\text{Abstract}|)$ regardless of the concrete space’s cardinality.

Three composition operators extend the framework: product-abstraction (component-wise), compose-abstraction (chaining two-level abstractions $C \xrightarrow{\alpha_1} M \xrightarrow{\alpha_2} A$), and abstract-conv-product (the full pipeline: abstract each component, verify FOSD abstractly, compose via convolution, lift to the concrete product). Validated postulate-free on infinite-state environments—Regional Policy ($\mathbb{N}$ states), Three-Zone ($\mathbb{N}$ states), and CartPole ($\mathbb{N}^4$ discretized, angle-based abstraction). Full implementation: `src/CSHRL/Core/Abstraction.agda`.

16 Learning as Symmetry Restoration

The Finder computes rankings at a fixed depth. But real learning is *incremental*: we observe samples, detect violations, and restore them.



Loop until convergence

Fig. 3. The learning loop: sample, test for violations, update ranking, refine depth.

16.1 Violations and Swaps

Suppose the current ranking at some state is [Stay, Go] (Stay preferred), but the Finder discovers at depth k that Go’s trace dominates Stay’s. This is a **violation**: the ranking disagrees with the evidence. The fix is to swap Go ahead of Stay, producing [Go, Stay]. Each swap fixes exactly one violated pair without disturbing unrelated pairs.

This is where the section title is literal: a swap of two actions is a *transposition*, and transpositions generate the symmetric group $S_{|\mathcal{A}|}$ (§3). Learning is therefore a walk through $S_{|\mathcal{A}|}$, applying transpositions until the ranking reaches the permutation that the environment’s

stream order dictates—restoring the correct symmetric structure one violated pair at a time.

Formally, a violation records which state, which two actions, and at what depth the disagreement was found:

The `DominanceOracle` is the ground-truth pairwise comparator—in practice, instantiated by the Finder’s trace comparison at the current depth. The fix is to **swap** the violated pair in the ranking:

16.2 Monotonicity Guarantee

The critical question is: does repeated swapping converge? The answer is yes, and we can bound the number of steps. The argument has three parts:

Theorem 2 (Swap Monotonicity). *Let v be a violation with actions (better, worse). After swapping: (1) the violated pair is correctly ordered; (2) unrelated pairs are unchanged; (3) total violations decrease by at least 1.*

In Agda, part (1) is `SwapFixesPair`: after swapping, the oracle confirms the pair is correctly ordered. Part (2) is `SwapPreservesUnrelated`: for any pair (a, b) where neither a nor b is involved in the swap, their relative order is unchanged. Together, these guarantee that each swap reduces the violation count by exactly one without introducing new violations.

Since each swap strictly decreases the violation count and violations are non-negative, the process terminates. The worst case is when every pair at every state is initially violated, giving a convergence bound:

$$\text{max-violations} \leq |S| \times \binom{|A|}{2} = |S| \times \frac{|A|(|A| - 1)}{2}$$

The `ConvergenceTheorem` packages the full guarantee: given a valid violation finder and a ground-truth oracle, iterated swapping from any initial ranking reaches zero violations in at most `max-violations` steps. Reading the type: the hypotheses require (i) `StrictDecrease`—each swap reduces the count; (ii) `FindViolValid`—the finder only reports genuine violations; (iii) oracle soundness—reported better actions truly dominate; and (iv) completeness—when no violation is found, the count is zero.

Learning is guaranteed to converge in at most `max-violations` swaps. Full proofs are in `src/CSHRL/Learning/Base.agda`.

Remark 3 (Full-ranking learning dissolves the exploration/exploitation trade-off). The convergence bound $|S| \times \binom{|A|}{2}$ counts *all* pairwise violations, including those between the two worst actions. To reach zero violations the learner must correctly order every pair—not just

identify the argmax. Uniform pairwise sampling suffices for convergence; no separate exploration mechanism (ϵ -greedy, entropy bonus, UCB) is needed.

16.3 Practical Learning Interface

The learning infrastructure provides a stateful interface supporting:

- **Checkpointing:** The `LearnerState` captures training progress; save to pause, resume by passing to `train-step`.
- **Incremental learning:** Process samples one at a time or in batches.
- **Active refinement:** The `ActiveLearner` maintains an explicit ranking updated on violation (swap), rather than re-querying the `Finder`.
- **Policy materialization:** A `PolicyTable` is a concrete list of (state, action-ranking) pairs—the verified analogue of trained weights. At deployment, lookup retrieves the cached ranking in $O(n)$; for states not in the table a fallback to the `Finder` guarantees no state is ever unhandled.

The same learning loop applies to both `CoinductiveHomomorphism` and `CoindHomo`: the only difference is whether the dominance oracle compares successor-value traces or action-value traces.

The output of this learning process is a *total ranking* at every state: every action has a definite position. This totality is exactly what enables the $O(1)$ adaptation described next—when an action becomes unavailable, we simply skip it in the ranking and pick the next one, with no recomputation needed.

16.4 Stochastic and Placement Learning

The learning infrastructure extends to both stochastic MDPs (comparing expected traces; module in `src/CSHRL/Learning/StochasticFiniteMDP.agda`) and `CombinatorialPlacementMDP` (short-circuit traces for absorbing Dead/Solved states; module in `src/CSHRL/Learning/CombinatorialPlacementMDP.agda`). All monotonicity guarantees carry over.

17 Instant Adaptation to Action Unavailability

In many practical settings—robotics, network routing, game playing—actions become unavailable at runtime. Classical RL stores only the argmax action and must recompute the policy at cost $O(|S| \cdot |A|)$ or worse.

Because the framework maintains a *total ranking* over all actions, adaptation is $O(1)$: when the top-ranked action becomes unavailable, the second-ranked action is already known.

Theorem 3 (Restricted Preservation). *If ranking R satisfies the homomorphism property for actions $\mathcal{A}$, and $\mathcal{A}' \subseteq \mathcal{A}$ is the set of available actions, then the restriction $R|_{\mathcal{A}'}$ satisfies the same property for $\mathcal{A}'$.*

The preservation property is pairwise: the fact that $a \leq_a b$ implies stream dominance does not depend on which other actions exist. Removing unavailable actions leaves the relative order of remaining actions unchanged.

Beyond filtering, actions can be **demoted** rather than removed—pushed to the end of the ranking so they are used only as a last resort: Classical RL stores only the argmax; when that action fails, the policy must be recomputed at cost $O(|S| \cdot |A|)$. The framework stores a total ranking, so when the top action fails the second-best is already available— $O(1)$ lookup.

This applies to both conditions: CoinductiveHomomorphism’s successor-value preservation and CoindHomo’s action-value preservation are both pairwise properties that restrict without loss.

18 Scope, Assumptions, and Limits

It is important to distinguish the *coinductive ideal* from the *algorithmic machinery*:

- **Coinductive optimality (ideal):** The definitions in §4 are about infinite streams and pointwise dominance at every time step.
- **Finder (approximation):** The Finder (§9) compares *finite* traces lexicographically at a fixed depth. It becomes exact under conditions verified by the Environment Class infrastructure (e.g., horizon sufficiency for placement problems).
- **Learner (policy discovery):** The Learner (§16) wraps Finder-style comparisons into a stateful learning loop.

The deterministic framework assumes: finite action space, deterministic step function, decidable reward order, and a horizon parameter for Finder approximation. Section 11 lifts to stochastic environments; Section 15 lifts to infinite state spaces.

A natural objection: the CoinductiveHomomorphism condition is undecidable in general. We view this as analogous to AIXI [5]: an uncomputable ideal that organizes a hierarchy of practical approximations. Crucially, Level ω (the full CoinductiveHomomorphism condition) is achievable for structured domains: the 8-Queens instance attains it by exploiting binary reward structure and finite game depth.

19 Verified Instantiations: From Theory to Tasks

The examples in this section validate that the formal machinery instantiates correctly. Each one that type-checks is a proof that the framework applies to that task class.

19.1 TwoState: Minimal Working Example

At Start, Go’s future is $(1, 1, 1, \dots)$ and Stay’s is $(0, 0, 0, \dots)$; the ranking $[Go, Stay]$ satisfies both CoinductiveHomomorphism and CoindHomo (the conditions coincide here since the immediate rewards are aligned). Full preservation proof in `src/CSHRL/Tasks/Verified/TwoState.agda`, with no postulates.

Beyond TwoState, a range of tasks instantiate the framework, each isolating a distinct phenomenon (Table 1). **Delayed Gratification** requires depth 2 to separate two paths with equal immediate reward ($GoA \rightarrow [0, 0]$ vs. $GoB \rightarrow [0, 1]$), illustrating depth-discovery, while **Trap Sparse** resolves at depth 1. **Key-Door-Treasure** exhibits a *state-dependent* ranking flip: before the key is held `GoRight` yields the blocked stream $[0, 0, 0, \dots]$, but once held it yields $[0, 0, 100, 100, \dots]$, inverting the ranking—a phase transition encoded directly in the traces rather than via gradual value propagation.

Task	Environment class	Phenomenon demonstrated
TwoState	FiniteDeterministic-MDP	minimal aligned case; postulate-free
Delayed Gratification	FiniteDeterministic-MDP	depth-discovery (depth 2 needed)
Trap Sparse	FiniteDeterministic-MDP	discrimination at depth 1
Key-Door-Treasure	FiniteDeterministic-MDP	state-dependent ranking flip
8-Queens	Combinatorial-PlacementMDP	scalable verification (8^8); postulate-free
BinarySacrifice, SkillInvestment, PreparationDilemma	FiniteDeterministic-MDP	separate the two conditions (§6)
CoinFlip, GamblersRuin, RandomWalk, BiasedBandit	StochasticFinite-MDP	lexicographic stochastic verification; postulate-free

Table 1. Verified instantiations across the three environment classes. All are machine-checked; classic tasks use the general `FiniteDeterministicMDP`, while structurally specialised classes discharge the bridge automatically.

19.2 8-Queens: Scalable Verification

The `Queens8Learning` module validates the full pipeline end-to-end: learning via `materialize-on-path` from the empty board, producing a

PolicyTable of eight entries. The following extract illustrates the interface; the full verified instantiation is in `src/CSHRL/Tasks/Verified/Queens8.agda`.

This confirms the materialized policy produces a valid non-attacking 8-Queens placement—verified end-to-end in Agda with `--safe` and zero postulates.

19.3 The Sacrifice Environments: Where the Conditions Separate

The three environments from Section 6—BinarySacrifice, SkillInvestment, and PreparationDilemma—are the verified instantiations that *separate* the two conditions. Each has a machine-checked CoinductiveHomomorphism proof and a machine-checked proof that no CoindHomo can rank the sacrificing actions. These live in `src/CSHRL/Tasks/Verified/BinarySacrifice.agda`, `src/CSHRL/Tasks/Verified/SkillInvestment.agda`, and `src/CSHRL/Tasks/Verified/PreparationDilemma.agda`.

20 Related Work

The framework connects to multiple research traditions but makes a distinct move: replacing scalar aggregation with a coinductive order on infinite futures, and distinguishing strategic evaluation (CoinductiveHomomorphism) from tactical evaluation (CoindHomo).

20.1 Lexicographic Multi-Objective RL

The most closely related line of work is **lexicographic multi-objective RL** (LMORL) [10,12]. LMORL orders *objectives* (different reward signals) lexicographically. Our framework orders *timesteps*: the same reward signal is compared pointwise at $t = 0$, then $t = 1$, then $t = 2$, forever.

- LMORL: “Safety reward first, then efficiency reward.”
- Our framework: “At every moment t , the better action’s future dominates.”

20.2 The Sacrifice Pattern in RL

The sacrifice pattern—accepting immediate cost for long-term benefit—is well-known but typically handled by increasing γ or via reward shaping. Our framework is the first to formally decouple optimality from immediate reward via a coinductive condition. CoinductiveHomomorphism judges actions solely by successor state quality, side-stepping the tension between immediate and future reward that plagues both discounted returns and CoindHomo.

Approach	Core idea	Relationship
Classical RL [11]	Optimize $\sum_t \gamma^t r_t$ via Bellman equations	CoindHomo <i>subsumes</i> this; CoinductiveHomomorphism subsumes successor return (§7)
Policy Gradients [16]	Gradient ascent on expected return	Optimization <i>method</i> ; our framework defines the <i>target</i>
Max-Entropy RL [4]	Entropy bonus for exploration	Full-ranking objective dissolves exploration/exploitation (Remark 3)
AIXI [5]	Universal prior over computable environments	Both are ideals; ours is <i>structurally</i> defined
Coalgebra [9]	Infinite behaviors via coinductive definitions	Direct inspiration: $\leq_s$ is a coinductive relation
MDP Homomorphisms [14]	Exploit state/action symmetries for efficiency	Different symmetry: theirs is state/action <i>automorphisms</i> for compression; ours is the symmetric group’s full <i>total order</i> on actions
Giry Monad [3]	Probability distributions as monad	Direct application: stochastic extension (§11)
CertRL [13]	Coq-verified RL convergence	Complementary: they verify classical RL; we verify coinductive structure

20.3 Constrained and Safe RL

Safe RL [2] treats some objectives as hard constraints rather than soft trade-offs. Constrained MDPs optimize a primary reward while ensuring constraint satisfaction (e.g., collision probability $< \epsilon$).

The framework’s relationship to safe RL is complementary:

- **Temporal constraints as dominance:** A safety constraint “never enter state X ” translates to: the safe action’s trace dominates at every timestep (the unsafe action’s trace eventually shows the penalty).
- **No constraint relaxation:** The coinductive order is strict—there is no ϵ -violation. If a ’s trace dominates b ’s at every step, the dominance is absolute.

20.4 Preference-Based, Ordinal, and Ranking RL

Preference-based RL (PbRL) [17] learns from pairwise comparisons rather than scalar rewards. This connects to our ranking structure, but the relationship is nuanced:

- PbRL *infers a latent scalar* from comparisons, then optimizes that scalar. Our framework requires only *ordinal structure*—a partial order on outcomes—and compares streams directly without scalar collapse.
- The Core module is parameterized by $_ \leq_r _ : \text{Reward} \rightarrow \text{Reward} \rightarrow \text{Set}$ —any partial order suffices. Human preference orderings are a valid instantiation.

Direct Preference Optimization (DPO) [8] bypasses explicit reward modelling by deriving an implicit scalar reward from pairwise preferences via the Bradley-Terry model, then optimizes a policy to maximize it. The pairwise structure of the input is discarded once the implicit reward is extracted. In our framework, by contrast, the pairwise ordering *is* the optimality criterion—preserved coinductively, not collapsed into a scalar.

Deep Ordinal Reinforcement Learning [18] modifies Q-learning for ordinal rewards via thresholded updates, mitigating overestimation in DQN. However, it retains discounts and bootstrapping, unlike the coinductive elimination of γ .

20.5 Coalgebraic Foundations

The coinductive stream order $\leq_s$ is a coalgebraic relation in the sense of Rutten [9]. Coalgebra provides the categorical semantics for infinite behaviors, and our formalization directly instantiates these ideas in Agda’s guarded coinduction. The coinductive keyword in the $\leq_s$ record is precisely the coalgebraic final coalgebra construction.

MDP homomorphisms [14] exploit state/action symmetries—group-theoretic automorphisms of the state/action space—to collapse equivalent states and reduce computation. Our use of “symmetric” is group-theoretic too, but on a different object: the symmetric group $S_{|\mathcal{A}|}$ acting on the action set, of which a total ranking is a single element (§3). The two notions are independent and could in principle be combined—an automorphism quotient on states with a full action ranking within each class. (Order-preservation between rankings and stream dominance is the *homomorphism* part of the framework, not the *symmetric* part; the two were previously conflated.)

20.6 Formal Verification of RL

CertRL [13] is a Coq library that machine-checks convergence proofs for value and policy iteration on finite MDPs, using the Giry monad for probabilistic reasoning. While CertRL provides rigorous guarantees for classical tabular RL, our framework extends formal verification to a different paradigm: coinductive structure preservation rather than scalar convergence. Both projects demonstrate that machine-verified RL theory is feasible; they target complementary foundations.

20.7 What Distinguishes This Work

- **Criterion vs. algorithm:** Classical RL and policy gradients are *algorithms* to optimize a scalar. Our framework defines an *optimality criterion*—a structural property that rankings may or may not satisfy.
- **Temporal structure:** The scalar return $\sum \gamma^t r_t$ erases timing; the coinductive order $\leq_s$ preserves it completely.
- **Pointwise dominance:** Unlike lexicographic MORL (priority among objectives) or average-reward RL (limit of partial sums), the framework requires dominance at *every* timestep.
- **Full rankings (symmetric):** Optimality is the entire total order on actions—an element of the symmetric group $S_{|\mathcal{A}|}$ —not the argmax . Ranking even the worst pair is required, which dissolves exploration/exploitation and makes adaptation to a lost top action $O(1)$.
- **Strategic vs. tactical:** CoinductiveHomomorphism judges actions solely by successor state quality; CoindHomo additionally demands immediate reward alignment. No prior work makes this decomposition.
- **Machine verification:** All results are fully implemented in Agda with type-checked proofs.

21 Future Work

- **Continuous distributions:** The stochastic framework (§11) represents distributions as `List (A × ℕ)`. Extending to measure-theoretic distributions would broaden applicability to continuous-outcome environments.
- **Continuous action spaces:** The framework assumes finite, enumerable actions. Extending to continuous action spaces (e.g., torque values) would require replacing enumeration-based ranking with optimization over compact sets.
- **Partially observable MDPs:** Lift rankings to belief-state distributions with stochastic transitions.
- **Multi-agent settings:** Rankings over joint action spaces with game-theoretic equilibrium conditions.
- **Program synthesis as policy representation:** The full-ranking objective (Remark 3) naturally extends to synthesized programs: a program satisfying all pairwise ranking constraints on observed states encodes sufficient environment structure to generalize to unseen states.

22 Conclusion

We have presented a framework that redefines optimality as *structure preservation* rather than scalar maximization. Two coinductive conditions capture two complementary notions of optimality:

- **CoinductiveHomomorphism** evaluates actions by the quality of the states they lead to—successor state quality. It is the strategic condition: an action that sacrifices immediate reward to reach a superior successor is correctly ranked.
- **CoindHomo** additionally requires that immediate transition rewards respect the ranking. It is the tactical condition: the better-ranked action must win both the immediate payoff and the long-term future.

The decomposition theorem establishes their precise relationship: CoindHomo equals CoinductiveHomomorphism plus immediate reward compatibility. For the vast majority of standard environments, these conditions coincide; three verified counterexamples demonstrate the strict gap.

The framework is supported by machine-verified results in Agda:

1. **Subsumption:** Both conditions imply classical argmax optimality for any finite horizon.
2. **Monotonic learning:** Violation-correction converges in at most $|S| \times \binom{|A|}{2}$ swaps.
3. **Instant adaptation:** Action unavailability requires $O(1)$ lookup, not full recomputation.
4. **Stochastic extension:** The framework lifts to probabilistic environments via the Girmonad, with lexicographic coinductive comparison.
5. **Distributional hierarchy:** FOSD and the SD[k] chain (§12-13) give parameterized verification strength for progressively broader utility function classes.
6. **Compositional algebra:** Verified rankings compose via product, sum, convolution, and scaling.
7. **State abstraction:** Rankings verified on finite abstract systems lift to arbitrary concrete spaces.
8. **Scalable verification:** The Environment Class architecture provides automatic trace-to-stream bridges; 8-Queens achieves a full CoindHomo for 8^8 candidate placements with zero postulates.

All Agda code shown in this paper is machine-checked: the literate source type-checks under Agda while producing this document.

23 Reproducibility and Build

- **Tested toolchain:** Agda 2.8.0, Agda Standard Library 2.0, Lua $\text{\LaTeX}$.
- **Build:** Compiling the literate source with `agda --latex type-checks` every embedded definition and emits the $\text{\LaTeX}$ rendered here; `lualatex` then produces the PDF. The full verified development accompanies this submission as supplementary material.
- **Fonts:** The document uses `fontspec` and Unicode math fonts (DejaVu, STIX Two Math).

References

1. Bellman, R.: Dynamic Programming. Princeton University Press (1957)
2. García, J., Fernández, F.: A comprehensive survey on safe reinforcement learning. *J. Mach. Learn. Res.* 16(42), 1437–1480 (2015)
3. Giry, M.: A categorical approach to probability theory. In: *Categorical Aspects of Topology and Analysis*. LNM, vol. 915, pp. 68–85. Springer (1982)
4. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *Proc. ICML*, pp. 1861–1870 (2018)
5. Hutter, M.: *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer (2005)
6. Li, L., Walsh, T.J., Littman, M.L.: Towards a unified theory of state abstraction for MDPs. In: *Proc. ISAIM* (2006)
7. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., et al.: Human-level control through deep reinforcement learning. *Nature* 518(7540), 529–533 (2015)
8. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: your language model is secretly a reward model. In: *Proc. NeurIPS* (2023)
9. Rutten, J.J.M.M.: Universal coalgebra: a theory of systems. *Theor. Comput. Sci.* 249(1), 3–80 (2000)
10. Skalse, J., Hammond, L., Griffin, C., Abate, A.: Lexicographic multi-objective reinforcement learning. In: *Proc. IJCAI*, pp. 3430–3436 (2022)
11. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*, 2nd edn. MIT Press (2018)
12. Tercan, H., Prabhu, S.: Thresholded lexicographic ordered multiobjective reinforcement learning. In: *Proc. ECAI* (2024)
13. Vajjha, K., Shinnar, A., Trager, B., Pestun, V., Fulton, N.: CertRL: formalizing convergence proofs for value and policy iteration in Coq. In: *Proc. CPP*, pp. 18–31. ACM (2021)
14. van der Pol, E., Worrall, D., van Hoof, H., Oliehoek, F., Welling, M.: MDP homomorphic networks: group symmetries in reinforcement learning. In: *Proc. NeurIPS* (2020)
15. Watkins, C.J.C.H., Dayan, P.: Q-learning. *Mach. Learn.* 8(3–4), 279–292 (1992)
16. Williams, R.J.: Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* 8(3–4), 229–256 (1992)

17. Wirth, C., Akrou, R., Neumann, G., Fürnkranz, J.: A survey of preference-based reinforcement learning methods. *J. Mach. Learn. Res.* 18(136), 1–46 (2017)
18. Zap, A., Chaudhuri, S., Topcu, U.: Deep ordinal reinforcement learning. In: *Proc. AAMAS*, pp. 1838–1846 (2019)